Parallel Processing Using MPI and OpenMP

Combined Lab Experiment Guide • Ubuntu Linux • C Programming

1. MPI — Message Passing Interface

Aim: To implement parallel processing using MPI and execute a program with multiple processes.

Introduction: MPI lets separate processes coordinate by exchanging messages. Each process has a rank (ID) and all processes belong to a communicator.

Step 1: Install MPICH and build tools

```
sudo apt update
sudo apt install -y mpich libmpich-dev build-essential
```

Check installation:

```
mpiexec --version
```

Step 2: Create the MPI source file

```
mkdir -p ~/parallel_lab
cd ~/parallel_lab
nano hello_mpi.c
```

Paste this code:

```
#include <stdio.h>
#include <mpi.h>

int main(int argc, char *argv[]) {
    int rank, size;

    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    printf("Hello from process %d out of %d processes\\n",
           rank, size);

    MPI_Finalize();
    return 0;
}
```

Save in Nano: Ctrl+O, Enter, then Ctrl+X.

Step 3: Compile and run MPI

```
mpicc hello_mpi.c -o hello_mpi
mpiexec -n 2 ./hello_mpi
mpiexec -n 4 ./hello_mpi
```

Example output for two processes (order may vary):

```
Hello from process 0 out of 2 processes
Hello from process 1 out of 2 processes
```

Key functions: MPI_Init initializes MPI; MPI_Comm_rank gets the process ID; MPI_Comm_size gets the total process count; MPI_Finalize ends MPI.

2. OpenMP — Open Multi-Processing

Aim: To implement parallel processing using OpenMP directives in C.

Introduction: OpenMP creates multiple threads within one process. Threads can share memory and run a parallel region concurrently.

Step 1: Install GCC and build tools

```
sudo apt update
sudo apt install -y build-essential
```

Check GCC:

```
gcc --version
```

Step 2: Create the OpenMP source file

```
mkdir -p ~/openmp_lab
cd ~/openmp_lab
nano parallel.c
```

Paste this code:

```
#include <stdio.h>
#include <omp.h>

int main(void) {
    #pragma omp parallel num_threads(4)
    {
        int id = omp_get_thread_num();
        printf("Hello from thread %d\\n", id);
    }

    printf("Parallel processing completed.\\n");
    return 0;
}
```

Save in Nano: Ctrl+O, Enter, then Ctrl+X.

Step 3: Compile and run OpenMP

```
gcc -fopenmp parallel.c -o parallel
./parallel
```

Expected output (thread order may vary):

```
Hello from thread 0
Hello from thread 2
Hello from thread 1
Hello from thread 3
Parallel processing completed.
```

Key statements: #pragma omp parallel starts a parallel region; num_threads(4) requests four threads; omp_get_thread_num() returns the current thread ID; -fopenmp enables OpenMP in GCC.

3. MPI vs OpenMP

MPI	OpenMP
Uses multiple processes.	Uses multiple threads within one process.
Processes commonly communicate by messages.	Threads share memory.
Compile with mpicc; run with mpiexec.	Compile with gcc -fopenmp; run directly.
Can work across multiple computers.	Typically used within a shared-memory computer.

Result

Parallel processing was demonstrated using both MPI processes and OpenMP threads. MPI was compiled and launched with MPICH, while OpenMP was compiled with GCC's -fopenmp option.

Troubleshooting

• If a command is missing, install the packages shown in the relevant section.
• Ensure you compile from the folder containing the source file.
• Output order may vary; this is normal for concurrent execution.
• MPI and OpenMP are separate approaches, so compile and run each program with its own commands.